pgBackRest — Production PostgreSQL Backup Solution

Table of Contents

1. Learning Objectives
2. Why pgBackRest
3. Architecture
4. Installation
5. pgbackrest.conf Configuration
6. Stanza Management
7. Running Backups
8. Restore Operations
9. PITR with pgBackRest
10. S3 Integration
11. Parallel Backup Performance
12. Backup Monitoring
13. Retention Management
14. Common Mistakes
15. Best Practices
16. Interview Questions
17. Exercises and Solutions

Learning Objectives

By the end of this module, you will be able to:

- Install and configure pgBackRest for production use
- Perform full, differential, and incremental backups
- Configure S3 integration for cloud backup storage
- Execute PITR to a specific timestamp or restore point
- Set up retention policies and monitoring
- Troubleshoot common pgBackRest issues

Why pgBackRest

```
PGBACKREST ADVANTAGES OVER pg_basebackup:

Feature                 | pg_basebackup | pgBackRest
------------------------------------------------------------
Incremental backups     | No            | Yes (delta)
Differential backups    | No            | Yes
Parallel backup         | No*           | Yes (multi-core)
S3/GCS/Azure native     | No            | Yes
WAL archiving           | Separate      | Built-in
Retention management    | Manual        | Automated
Backup verification     | Manual        | Built-in --verify
Encryption              | OS-level only | Built-in AES-256
```

```
Compression             | Basic         | Multiple (lz4, zst, gz, bz2)
Resume failed backup    | No            | Yes
Repository backup       | Single        | Multiple (local + S3)

* pg_basebackup supports parallel only for pg_dump-style
  pgBackRest is significantly more capable for production use.
```

Architecture

```
PGBACKREST DEPLOYMENT:

┌──────────────────┐     archive-push/get    ┌──────────────────────────────┐
│  PostgreSQL      │ ──────────────────────> │   pgBackRest Repository      │
│  Primary         │                         │   ┌──────────────────┐       │
│  + pgBackRest    │ ─── backup-push ──────> │   │  Backups:        │       │
│  agent           │                         │   │  full/diff/incr  │       │
└──────────────────┘                         │   │                  │       │
                                             │   │  WAL Archive:    │       │
┌──────────────────┐     archive-get         │   │  000001...       │       │
│  PostgreSQL      │ <────────────────────── │   │  000002...       │       │
│  Standby         │ (restore from archive)  │   └──────────────────┘       │
│                  │                         │                              │
└──────────────────┘                         │   STORAGE: NFS, S3,          │
                                             │    Azure, GCS                │
┌──────────────────┐                         └──────────────────────────────┘
│  Backup Host     │ ─── backup-push ──────>│ (optional dedicated
│  (optional       │     restore             │  backup host)
│   repository     │                         │
│   host)          │                         │
└──────────────────┘
```

Installation

```bash
# Ubuntu/Debian
sudo apt-get install -y pgbackrest

# RHEL/CentOS
sudo yum install -y pgbackrest

# From source
git clone https://github.com/pgbackrest/pgbackrest.git
cd pgbackrest
# Follow build instructions for your platform

# Verify
pgbackrest version
# pgBackRest 2.49
```

---

pgbackrest.conf Configuration

```
# /etc/pgbackrest/pgbackrest.conf

#———————————————————————————————————————
# GLOBAL SETTINGS
#———————————————————————————————————————
[global]

# Repository: local storage (single server)
repo1-path=/var/lib/pgbackrest
repo1-retention-full=2
repo1-retention-diff=14
repo1-retention-archive=14          # Keep 14 days of WAL archives
repo1-retention-archive-type=diff   # WAL retained relative to diff backups

# Repository: S3 (add as second repository)
repo2-type=s3
repo2-s3-bucket=my-pg-backups
repo2-s3-region=us-east-1
repo2-s3-endpoint=s3.amazonaws.com
repo2-path=/production
repo2-retention-full=4
repo2-retention-archive=30

# Compression
compress-type=lz4               # lz4: fast; zst: best ratio; gz: compatible
compress-level=6                # 1=fastest, 9=smallest

# Parallelism
process-max=4                   # Use 4 processes for backup/restore/archive

# Logging
log-level-console=info
log-level-file=detail
log-path=/var/log/pgbackrest

# Start fast checkpoint
start-fast=y

# Delta restore (only copy changed files)
delta=y

# Verify backup integrity
backup-standby=y                # Take backup from standby if available

#———————————————————————————————————————
# STANZA SETTINGS (one per PostgreSQL cluster)
#———————————————————————————————————————
[mydb]
```

```
pg1-path=/var/lib/postgresql/16/main
pg1-port=5432
pg1-user=postgres
pg1-socket-path=/var/run/postgresql

# For remote database (repository on separate host):
# pg1-host=db.example.com
# pg1-host-user=postgres

# Standby (take backups from standby):
pg2-path=/var/lib/postgresql/16/main
pg2-host=standby.example.com
pg2-host-user=postgres
```

Archive-Only Configuration (for PostgreSQL side)

```
# postgresql.conf
archive_mode = on
archive_command = 'pgbackrest --stanza=mydb archive-push %p'
archive_timeout = 300

# For restore:
restore_command = 'pgbackrest --stanza=mydb archive-get %f %p'
```

Stanza Management

A "stanza" is pgBackRest's term for a configured PostgreSQL cluster.

```
# Create the stanza (must run on repository host or same server)
sudo -u postgres pgbackrest --stanza=mydb stanza-create

# Verify configuration and connectivity
sudo -u postgres pgbackrest --stanza=mydb check
# Output:
# P00   INFO: check command begin 2.49: --exec-id=... --pg1-path=... --stanza=mydb
# P00   INFO: check repo1 configuration (primary)
# P00   INFO: check repo1 archive for WAL (primary)
# P00   INFO: WAL segment 000000010000000000000001 successfully stored
# P00   INFO: check command end: completed successfully

# Get info about a stanza
sudo -u postgres pgbackrest --stanza=mydb info

# Delete a stanza (removes all backups and archive for that stanza)
sudo -u postgres pgbackrest --stanza=mydb stanza-delete --force
```

Running Backups

```
#──────────────────────────────────────────────────────────
# FULL BACKUP
#──────────────────────────────────────────────────────────
sudo -u postgres pgbackrest --stanza=mydb backup --type=full

# With specific options:
sudo -u postgres pgbackrest \
    --stanza=mydb \
    --type=full \
    --compress-type=lz4 \
    --process-max=4 \
    --start-fast=y \
    backup

#──────────────────────────────────────────────────────────
# DIFFERENTIAL BACKUP (changes since last full)
#──────────────────────────────────────────────────────────
sudo -u postgres pgbackrest --stanza=mydb backup --type=diff

#──────────────────────────────────────────────────────────
# INCREMENTAL BACKUP (changes since last any backup)
#──────────────────────────────────────────────────────────
sudo -u postgres pgbackrest --stanza=mydb backup --type=incr

#──────────────────────────────────────────────────────────
# BACKUP FROM STANDBY
#──────────────────────────────────────────────────────────
# Configure pg2-host in pgbackrest.conf, then:
sudo -u postgres pgbackrest --stanza=mydb backup \
    --type=full \
    --backup-standby   # Use standby if available

#──────────────────────────────────────────────────────────
# CHECK BACKUP STATUS
#──────────────────────────────────────────────────────────
sudo -u postgres pgbackrest --stanza=mydb info

# Output example:
# stanza: mydb
#     status: ok
#     cipher: none
#
#     db (current)
#         wal archive min/max (16): 000000010000000000000001/000000010000000000000025
#
#         full backup: 20240115-020000F
#             timestamp start/stop: 2024-01-15 02:00:00+00 / 2024-01-15 02:45:32+00
#             wal start/stop: 000000010000000000000001 / 000000010000000000000003
#             database size: 10.5GB, database backup size: 10.5GB
#             repo1: backup size: 3.2GB
#
```

```
#         diff backup: 20240116-020000F_20240116-020000D
#             timestamp start/stop: 2024-01-16 02:00:00+00 / 2024-01-16 02:15:22+00
#             database size: 10.6GB, database backup size: 512MB (changed)
#             repo1: backup size: 182MB
```

Scheduled Backups via Cron

```
# /etc/cron.d/pgbackrest
SHELL=/bin/bash
PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin

# Full backup: every Sunday at 01:00
0 1 * * 0 postgres pgbackrest --stanza=mydb backup --type=full 2>&1 | logger -t
pgbackrest-full

# Differential backup: Mon-Sat at 01:00
0 1 * * 1-6 postgres pgbackrest --stanza=mydb backup --type=diff 2>&1 | logger -t
pgbackrest-diff

# Note: WAL archiving is continuous (archive_command, not cron)
```

Restore Operations

```
# STOP PostgreSQL before restore!
sudo systemctl stop postgresql@16-main

#──────────────────────────────────────────────────────
# FULL RESTORE (latest backup)
#──────────────────────────────────────────────────────

# Method 1: Restore to original location
sudo -u postgres pgbackrest --stanza=mydb --delta restore
# --delta: only restore changed files (fast for minor differences)

# Method 2: Clean restore (clears everything)
sudo -u postgres pgbackrest --stanza=mydb restore

# Method 3: Restore from specific backup label
sudo -u postgres pgbackrest --stanza=mydb \
    --set=20240115-020000F restore

# Start PostgreSQL after restore
sudo systemctl start postgresql@16-main

#──────────────────────────────────────────────────────
# RESTORE TO DIFFERENT DIRECTORY
#──────────────────────────────────────────────────────
sudo -u postgres pgbackrest --stanza=mydb restore \
    --pg1-path=/var/lib/postgresql/restore_test
```

```
# Then configure the restored instance with different port
# and start as a test instance

#──────────────────────────────────────────────────────
# VERIFY RESTORE
#──────────────────────────────────────────────────────
sudo -u postgres pgbackrest --stanza=mydb restore --log-level-console=detail
# Shows which files were restored and any issues
```

PITR with pgBackRest

```
# Restore to a specific point in time

sudo systemctl stop postgresql@16-main

# Restore with time target
sudo -u postgres pgbackrest --stanza=mydb restore \
    --type=time \
    --target="2024-01-15 14:31:00+00" \
    --target-action=promote \   # Promote to primary after reaching target
    --delta

# OR: restore to a named restore point
sudo -u postgres pgbackrest --stanza=mydb restore \
    --type=name \
    --target="before_migration" \
    --target-action=pause

# Start PostgreSQL (it enters recovery mode automatically)
sudo systemctl start postgresql@16-main

# Monitor recovery
sudo tail -f /var/log/postgresql/postgresql-16-main.log

# If target-action=pause: manually promote after verification
sudo -u postgres psql -c "SELECT pg_wal_replay_resume();"

# Check recovery status
sudo -u postgres psql -c "SELECT pg_is_in_recovery();"
```

S3 Integration

```
# /etc/pgbackrest/pgbackrest.conf — S3 Configuration

[global]
# S3 Repository
repo1-type=s3
```

```
repo1-s3-bucket=my-pg-backups-prod
repo1-s3-region=us-east-1
repo1-s3-endpoint=s3.amazonaws.com
repo1-path=/production-db

# Authentication (choose one):
# Option A: IAM instance role (recommended for EC2 - no credentials!)
# Nothing needed — AWS SDK picks up instance role automatically

# Option B: Environment variables
# AWS_ACCESS_KEY_ID and AWS_SECRET_ACCESS_KEY set in environment

# Option C: Explicit in config (not recommended - credentials in file)
# repo1-s3-key=AKIAIOSFODNN7EXAMPLE
# repo1-s3-key-secret=wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY

# Encryption for backup data
repo1-cipher-type=aes-256-cbc
repo1-cipher-pass=encryption_passphrase_here   # Store in vault!

# S3 URIs style
repo1-s3-uri-style=host      # or 'path' for S3-compatible (MinIO, etc.)

# GCS (Google Cloud Storage)
# repo1-type=gcs
# repo1-gcs-bucket=my-pg-backups
# repo1-gcs-key=/etc/pgbackrest/gcs-key.json

# Azure Blob Storage
# repo1-type=azure
# repo1-azure-account=mystorageaccount
# repo1-azure-container=pgbackups
# repo1-azure-key=base64_encoded_key
```

```
# Test S3 connectivity
sudo -u postgres pgbackrest --stanza=mydb check
# Should complete without errors

# Verify backup is in S3
aws s3 ls s3://my-pg-backups-prod/production-db/

# S3 lifecycle policies (managed via AWS console or CLI):
aws s3api put-bucket-lifecycle-configuration \
    --bucket my-pg-backups-prod \
    --lifecycle-configuration file://s3-lifecycle.json
# Example: Move to Glacier after 30 days, delete after 7 years
```

Parallel Backup Performance

```
# pgbackrest.conf - Tune for performance

[global]
# Parallel processes (set to number of CPU cores, max ~8 typically)
process-max=8

# Compression: lz4 is fastest, zst has best ratio
compress-type=lz4
compress-level=1       # 1=fastest, trades size for speed

# IO settings
buffer-size=8MB       # Buffer size for transfers

# Archive parallel push (requires archive-push-queue-max)
archive-push-queue-max=4GB  # Queue for async WAL push
```

```
# Benchmark backup with different settings
time sudo -u postgres pgbackrest --stanza=mydb --process-max=1 backup --type=full
time sudo -u postgres pgbackrest --stanza=mydb --process-max=4 backup --type=full
time sudo -u postgres pgbackrest --stanza=mydb --process-max=8 backup --type=full

# Compare sizes with different compression
pgbackrest --stanza=mydb --compress-type=lz4 backup --type=full  # Fast, ~2x
compression
pgbackrest --stanza=mydb --compress-type=zst backup --type=full  # Slow, ~4x
compression
```

Backup Monitoring

```
# Quick status check
sudo -u postgres pgbackrest --stanza=mydb info --output=json | python3 -m json.tool

# Check backup freshness
sudo -u postgres pgbackrest --stanza=mydb info | grep "timestamp stop"

# Verify backup integrity
sudo -u postgres pgbackrest --stanza=mydb verify
# Checks:
# - Backup files exist and are readable
# - WAL archive is complete (all required segments present)
# - Checksums match

# Integration with monitoring (Prometheus/Grafana):
# Use pgbackrest_exporter or custom script:
#!/bin/bash
BACKUP_AGE=$(pgbackrest --stanza=mydb info --output=json | \
    python3 -c "
import json, sys
```

```
from datetime import datetime
data = json.load(sys.stdin)
backups = data[0]['backup']
if backups:
    last = backups[-1]
    stop_time = datetime.strptime(last['timestamp']['stop'], '%Y-%m-%d %H:%M:%S%z')
    age = (datetime.now(stop_time.tzinfo) - stop_time).total_seconds() / 3600
    print(f'{age:.1f}')
else:
    print(9999)
")
echo "pgbackrest_backup_age_hours $BACKUP_AGE"
```

Retention Management

```
# pgbackrest.conf retention settings

[global]
# Full backup retention
repo1-retention-full=2          # Keep 2 full backups (delete older)
repo1-retention-full-type=count  # count or time

# Differential backup retention (relative to retained fulls)
repo1-retention-diff=14         # Keep 14 differentials

# WAL archive retention
repo1-retention-archive=14      # Keep 14 days of WAL
repo1-retention-archive-type=diff  # WAL retained by relationship to diff backups
```

```
# Run expire (cleanup based on retention policy)
sudo -u postgres pgbackrest --stanza=mydb expire

# Show what would be expired (dry run)
sudo -u postgres pgbackrest --stanza=mydb expire --dry-run

# Force expire to specific backup (DANGEROUS - data loss)
sudo -u postgres pgbackrest --stanza=mydb expire \
    --set=20240101-020000F \     # Expire all backups before this one
    --force
```

Common Mistakes

1. **Not running `stanza-create`** before first backup
2. **Not configuring `archive_command`** — WAL not archived, PITR broken
3. **Wrong PostgreSQL data directory** in `pg1-path`
4. **S3 credentials not configured** — backups fail silently
5. **Not running `pgbackrest check`** after setup
6. **No encryption** for backups containing sensitive data

7. **Single repository** — no offsite copy
8. **Not testing restore** — discovering backup is unusable during DR
9. **process-max too high** — consumes all I/O, impacts production
10. **Retention too aggressive** — can't recover past 7 days when needed

---

Best Practices

1. **Always run `pgbackrest check`** after any configuration change
2. **Use two repositories** (local + S3) for redundancy
3. **Enable encryption** (`repo-cipher-type = aes-256-cbc`)
4. **Backup from standby** when possible (`--backup-standby`)
5. **Verify backups regularly** (`pgbackrest verify`)
6. **Test restores monthly** to a separate environment
7. **Use lz4 compression** for best speed/ratio balance in most cases
8. **Monitor backup age** and alert if last backup is > 25 hours old
9. **Keep retention > RPO requirement** — if RPO = 24h, keep at least 2 full backups
10. **Document the restore procedure** with pgBackRest commands

---

Interview Questions

Q1: What is a pgBackRest stanza?

A: A stanza is a named configuration in pgbackrest.conf that represents a specific PostgreSQL cluster. It defines the PostgreSQL data directory, connection settings, and links to one or more backup repositories. Each PostgreSQL cluster needs its own stanza. The stanza name is used in all pgBackRest commands (`--stanza=mydb`).

Q2: What are the three backup types in pgBackRest?

A: Full: complete backup of all database files. Differential: files changed since the last full backup. Incremental: files changed since the last backup (any type). Full requires the most space; incremental requires the least. Restore from differential needs full + differential; restore from incremental needs full + all incrementals.

Q3: How is pgBackRest different from pg_basebackup?

A: pgBackRest adds: incremental/differential backups (pg_basebackup is full-only), parallel backup/restore across multiple CPU cores, built-in WAL archiving management, native S3/GCS/Azure support, built-in encryption, retention management, delta restore, and backup verification. pg_basebackup is simpler but lacks these production features.

Q4: How does delta restore work?

A: Delta restore (`pgbackrest restore --delta`) compares checksums of files in the existing data directory against the backup. Only files that differ are copied. This dramatically speeds up restore when only a small portion of data changed, avoiding copying the entire database. It's safe because checksums guarantee correctness.

Q5: How do you configure pgBackRest to use multiple repositories?

A: In pgbackrest.conf, define `repo1-*` and `repo2-*` settings. pgBackRest will write backups and WAL archives to ALL configured repositories simultaneously. This provides redundancy: local repo for fast restore, S3 repo for offsite DR. Each repo can have independent retention policies.

Q6: What is `pgbackrest verify` and why is it important?

A: `pgbackrest verify` checks all backup sets and WAL archives in the repository for integrity: (1) Files exist in expected locations. (2) File checksums match what was recorded during backup. (3) WAL archive is complete (no gaps between backup start and current time). It should be run regularly to detect storage corruption before it's needed for recovery.

Q7: How do you perform PITR using pgBackRest?

A: (1) Stop PostgreSQL. (2) Run: `pgbackrest --stanza=mydb restore --type=time --target="2024-01-15 14:31:00" --delta`. (3) Start PostgreSQL — it enters archive recovery mode automatically. (4) PostgreSQL fetches WAL via restore_command (`pgbackrest archive-get`) and replays to the target. (5) Verify data, then promote.

Exercises and Solutions

Exercise 1: Set Up Full pgBackRest Environment

```
#!/bin/bash
# setup_pgbackrest.sh

# 1. Create repository directory
mkdir -p /var/lib/pgbackrest
chown postgres:postgres /var/lib/pgbackrest
chmod 750 /var/lib/pgbackrest

# 2. Create log directory
mkdir -p /var/log/pgbackrest
chown postgres:postgres /var/log/pgbackrest

# 3. Configure pgbackrest.conf
cat > /etc/pgbackrest/pgbackrest.conf << 'EOF'
[global]
repo1-path=/var/lib/pgbackrest
repo1-retention-full=2
repo1-retention-diff=7
repo1-retention-archive=14
compress-type=lz4
process-max=4
start-fast=y
log-level-console=info
log-level-file=detail

[production]
pg1-path=/var/lib/postgresql/16/main
pg1-port=5432
EOF

# 4. Configure PostgreSQL archiving
sudo -u postgres psql << 'SQL'
ALTER SYSTEM SET archive_mode = 'on';
```

```
ALTER SYSTEM SET archive_command = 'pgbackrest --stanza=production archive-push %p';
ALTER SYSTEM SET archive_timeout = '300';
SQL

# 5. Restart PostgreSQL
sudo systemctl restart postgresql@16-main

# 6. Create stanza
sudo -u postgres pgbackrest --stanza=production stanza-create

# 7. Verify
sudo -u postgres pgbackrest --stanza=production check

# 8. Take first full backup
sudo -u postgres pgbackrest --stanza=production backup --type=full

# 9. Show info
sudo -u postgres pgbackrest --stanza=production info

echo "pgBackRest setup complete!"
```

Cross-References

- 05_wal_archiving.md — WAL archiving concepts
- 04_pitr.md — PITR using pgBackRest
- 07_disaster_recovery.md — DR scenarios
- 08_backup_monitoring.md — Monitoring backups